



Cquirrel Project Report

Validation of Continuous Query Processing on TPC-H Q1 and Q16

MSBD 6910A Project

Project Type: Individual Project

WU Biyang

Abstract

Cquirrel is a demonstration system for continuous query processing over acyclic relational schemas. It accepts SQL or a JSON query plan, compiles the plan into an Apache Flink program, and maintains query answers incrementally over insertion and deletion streams. This report evaluates the system on two representative TPC-H query types: Q1, a single-relation group-by aggregate over `lineitem`, and Q16, an acyclic join query over `part`, `partsupp`, and `supplier`. The validation results support expected behavior under insertion-only and sliding-window streams. Q1 exposes strict floating-point tolerance effects at larger sliding-window settings, while Q16 passes scale factor 0.1 validation for both stream types.

Project repository used in this report: The source code, data, and other artifacts have been made available at <https://github.com/Ha0504/Cquirrel-release>.

1 Background of the Project and Related Work

Modern analytical applications often evaluate queries over continuously changing data. Instead of answering a static query once, a stream processor must apply base-table insertions and deletions and emit the resulting changes to the query answer. This problem is commonly studied as continuous query evaluation or incremental view maintenance (IVM).

Cquirrel, short for Continuous query processing over acyclic relational schemas, is a PVLDB 2021 demonstration system by HKUST and Alibaba researchers [1]. Its target workload is SPJA queries whose primary-key and foreign-key dependencies form a directed acyclic graph (DAG). This restriction is central: it allows Cquirrel to maintain multi-way foreign-key joins holistically rather than decomposing every query into binary joins. The theoretical basis is Wang and Yi’s SIGMOD 2020 work on maintaining acyclic foreign-key joins under updates [2]. Their enclosure parameter λ characterizes update sequences; insertion-only and FIFO/sliding-window streams have $\lambda = 1$, yielding $O(\lambda)$ amortized update time. Cquirrel therefore complements systems such as DBToaster, Dynamic Yannakakis, Trill, and Flink’s native stateful streaming runtime by compiling acyclic relational queries into Flink operators that maintain the whole join structure directly.

2 Description of the Query and the Algorithm

2.1 Supported Query Class

The project is specialized for TPC-H-style schemas and queries. A supported query can contain selections, projections, joins, and aggregations. Join predicates must follow primary-key to foreign-key relationships, and the corresponding graph must be acyclic. The Cquirrel paper notes that the TPC-H schema satisfies this form, and the release uses TPC-H data and query templates as the demonstration workload.

The default output model is delta enumeration. Let db denote the current database state and $Q(db)$ denote the current query answer. When an update u , either an insertion or deletion, arrives, the system updates its maintained state from $D(db)$ to $D(db + u)$ and emits ΔQ , the difference between $Q(db)$ and $Q(db + u)$. For a group-by aggregate, this means the output stream contains updated aggregate values for affected groups rather than a full recomputation of all groups.

The evaluation uses two TPC-H query types. Q1 is a single-relation aggregation. The implemented SQL is:

```
select
    l_returnflag,
    l_linestatus,
    sum(l_quantity) as sum_qty,
    sum(l_extendedprice) as sum_base_price,
    sum(l_extendedprice*(1-l_discount)) as sum_disc_price,
    sum(l_extendedprice*(1-l_discount)*(1+l_tax)) as sum_charge,
    avg(l_quantity) as avg_qty,
    avg(l_extendedprice) as avg_price,
    avg(l_discount) as avg_disc,
    count(*) as count_order
from
    lineitem
where
    l_shipdate <= date "1998-09-02"
group by
    l_returnflag,
    l_linestatus
```

It covers SUM, AVG, COUNT, arithmetic expressions, filtering, and GROUP BY without join-state propagation. Q16 is an acyclic join aggregation. The implemented SQL is:

```
select
    p_brand, p_type, p_size,
    count(distinct ps_suppkey) as supplier_cnt
from
    part, partsupp, supplier
where
    p_partkey = ps_partkey
    and ps_suppkey = s_suppkey
    and p_brand <> 'Brand#45'
    and p_type not like 'MEDIUM POLISHED%'
    and p_size in (49, 14, 23, 45, 19, 3, 36, 9)
    and s_comment not like '%Customer%Complaints%'
group by
    p_brand, p_type, p_size
```

Here the result depends on a three-relation foreign-key join, two join predicates, four selection predicates, and a grouped distinct supplier count. Each base update may affect join liveness before reaching the aggregate, so Q16 exercises relation process functions, side-output routing, foreign-key indexes, and the aggregate process function together. Thus Q1 and Q16 form complementary query categories for evaluating the same Cquirrel pipeline.

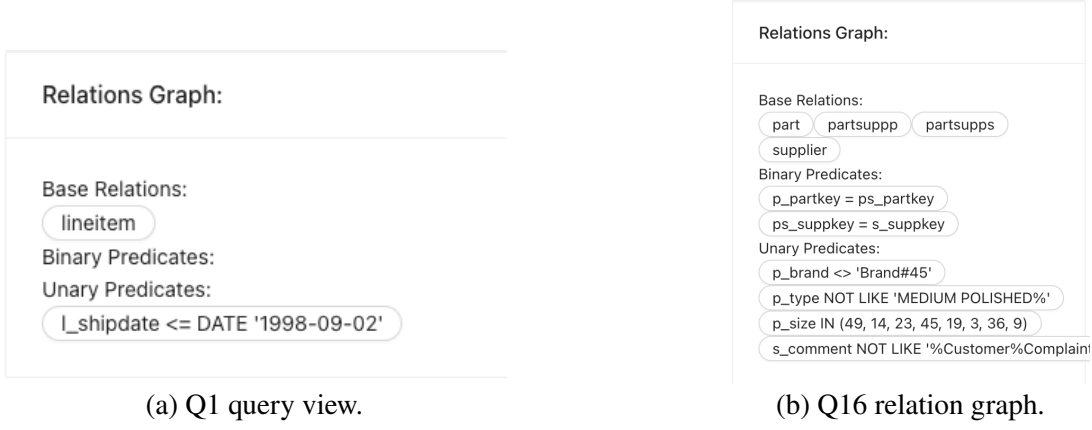


Figure 1: Logical views for the two evaluated query types. Q1 is a single-relation aggregate, whereas Q16 contains an acyclic join over `part`, `partsupp`, and `supplier`.

2.2 Algorithmic Flow

Cquirrel’s algorithm can be understood as incremental maintenance over a foreign-key DAG. Each base tuple has a sign, +1 for insertion and −1 for deletion. For every relation, the generated process function stores keyed state for tuples and for the liveness counters needed by its parent and child relations. When a new update arrives, Cquirrel does not recompute $Q(db)$. It updates only the affected keyed state, computes the delta caused by that update, and propagates the delta along the acyclic join graph. For aggregates, the generated aggregate process function applies the signed contribution to the target group:

$$S_g \leftarrow S_g + \text{sign}(u) \cdot \text{value}(u), \quad C_g \leftarrow C_g + \text{sign}(u),$$

where S_g is a maintained sum-like state and C_g is the count state for group g . AVG fields are then derived from the maintained sums and counts. In the multi-relation case, the same idea is applied after the update has been checked against selection predicates and foreign-key dependencies. The acyclic structure is what makes this propagation bounded: for insertion-only and FIFO/sliding-window streams, the enclosure parameter λ is 1, so the theoretical amortized update cost is $O(\lambda)$.

At the implementation level, Cquirrel parses SQL or JSON, constructs the foreign-key DAG, generates relation and aggregate process functions, dispatches signed records to relation-specific side outputs, maintains keyed relation and aggregate state, and emits result deltas. Figure 2 shows the generated flows for the two evaluated query types.

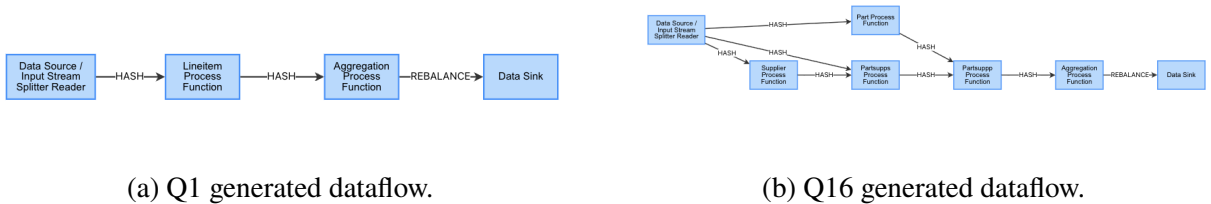


Figure 2: Generated Flink dataflows for the two evaluated query types. Q1 routes `lineitem` updates directly to an aggregate process function; Q16 first maintains relation process functions for the acyclic join and then updates the grouped supplier-count aggregate.

The algorithm differs from a conventional plan of binary joins. A standard plan joins two relations, materializes or streams an intermediate result, and then joins that intermediate with another relation.

Under high-rate updates, one base-tuple update may create many intermediate changes. Cquirrel instead maintains indexes over the whole foreign-key join structure and propagates liveness through the DAG. A tuple is live when it contributes to the current join result; counters track whether all required foreign-key dependencies are live. This lets the system enumerate only affected delta tuples. The update cost depends on enclosure λ ; FIFO and sliding-window streams have $\lambda = 1$, which explains why Cquirrel is well suited to continuously sliding analytical windows.

3 Implementation Details

The release is organized as a demonstration stack rather than a single library. The main modules are summarized in Table 1. The code generator is the central component: `codegen/src/main/java/CodeGen.java` accepts SQL or JSON, builds the query representation, writes Scala relation and aggregate process functions, creates the Flink job, and packages the generated project.

Path	Responsibility
DemoTools/ DataGenerator codegen	Converts TPC-H .tbl files into Cquirrel input streams with insert and delete tags. Java command-line tool that parses SQL or JSON and generates a Scala/Flink project plus a jar.
gui/cquirrel_flask and gui/cquirrel_react	Backend and frontend for query input, code generation, Flink submission, result display, and validation summaries.

Table 1: Main modules in the Cquirrel release.

The generated job uses the Flink DataStream API. It reads tagged update records, converts rows into Cquirrel Payload objects, dispatches relation-specific records through side outputs, and wires generated operators with `keyBy`, `connect`, and `process`. For Q1, the job filters `lineitem` and updates one aggregate process function. For Q16, it creates relation process functions for `part`, `partsupp`, and `supplier`, then sends live join tuples to the grouped supplier-count aggregate.

4 Experimental Setup and Data Sets Used

The experiments use the released Cquirrel stack: a Java/Scala code generator, Apache Flink as the streaming engine, and a Python/React validation interface. The original release targets Flink 1.11.2 and Java 8. Workloads are derived from TPC-H base tables and encoded as signed update streams. Insert-only streams write each tuple once with a positive tag, while sliding-window streams later emit negative records to model tuple expiration. The evaluated settings are summarized in Table 2.

Query	Scale factor	Stream type	Validation role
Q1	0.01	insert_only	small insert-only aggregate
Q1	0.01, 0.2, 0.3	sliding_windows	aggregate under tuple expiration
Q16	0.1	insert_only	acyclic join aggregate
Q16	0.1	sliding_windows	join aggregate after expiration

Table 2: Evaluated query and stream settings.

This report focuses on Q1 and Q16 because they cover two distinct query structures: single-relation aggregation and acyclic join aggregation. For each run, the generated Flink output is compared with an independently computed reference result. The validation output is reported at metric level, so a query with multiple aggregate fields contributes one comparison per group and per metric.

The evaluation covers two structural query types. Q1 validates single-relation aggregate maintenance over `lineitem`, while Q16 validates acyclic join aggregation over `part`, `partsupp`, and `supplier`. Figure 3 places representative Q1 and Q16 result views together.

Q16 Validation: PASS			
 Matched 2762 / 2762 metrics Scale factor 0.1, stream type insert_only			
P_BRAND	P_TYPE	P_SIZE	SUPPLIER_CNT
Brand#14	SMALL ANODIZED NICKEL	45	12

(b) Q16 result and validation view.

For Q1, the validator computes the same metrics independently using Python Decimal arithmetic and compares Cquirrel output against the reference at field level. Let a be the generated value and e the reference. For numeric metrics other than COUNT_ORDER, the default condition is:

$$|a - e| \leq 0.00005 \quad \text{or} \quad \frac{|a - e|}{|e|} \leq 0.$$

Thus the practical tolerance is the absolute error bound 5×10^{-5} , unless the value is exactly equal; COUNT_ORDER must match exactly. The Q1 passing rate is the number of matched metrics divided by the number of groups times eight metrics. For Q16, each group has one reported supplier-count metric. The insert-only Q16 run at scale factor 0.1 therefore reports 2762 matched comparisons over 2762 output groups, while the sliding-window run ends with an empty materialized state and has no remaining expected groups.

[illegible]

(b) Q1 0.01 sliding-window.

[illegible]

(d) Q1 0.3 sliding-window.

Q16 Validation: PASS

✓

Matched 0 / 0 metrics
Scale factor 0.1, stream type sliding_windows

Actual Output

(f) O16 0.1 sliding-window.

Figure 4: Validation evidence for all evaluated Q1 and Q16 stream settings. Q1 covers single-relation aggregation; Q16 covers acyclic join aggregation.

Query type	Scale factor	Stream type	Matched metrics	Total metrics	Passing rate	Result
Q1 (single-relation aggregate)	0.01	insert_only	32	32	100.00%	PASS
Q1	0.01	sliding_windows	32	32	100.00%	PASS
Q1	0.2	sliding_windows	29	32	90.62%	PARTIAL PASS
Q1	0.3	sliding_windows	30	32	93.75%	PARTIAL PASS
Q16 (acyclic join aggregate)	0.1	insert_only	2762	2762	100.00%	PASS
Q16	0.1	sliding_windows	0	0	N/A	PASS

Table 3: Observed validation results for the evaluated query types. Q1 covers single-relation aggregation; Q16 covers acyclic join aggregation. The Q16 sliding-window row has no denominator because the final materialized answer is empty after tuple expiration.

Table 3 summarizes the validation evidence across both query types. Q1 evaluates numerical aggregate maintenance. The two scale factor 0.01 runs pass all 32 metric comparisons, while the 0.2 and 0.3 sliding-window runs obtain 90.62% and 93.75%. These partial passes are caused by residual floating-point values after repeated additions and subtractions; the group keys and counts remain structurally consistent with the final-window reference. The mismatch is therefore best interpreted as a strict numerical-tolerance effect rather than a failure of the generated streaming plan. Q16 evaluates acyclic join aggregation: the scale factor 0.1 insertion-only run produces 2762 grouped supplier-count rows, all matching the reference, and the sliding-window run correctly converges to an empty final answer after deletions cancel the inserted tuples.

The generated plans are consistent with these query categories. Figures 1 and 2 show that the Q1 path has one relation process feeding one aggregate process, whereas Q16 splits the stream into `part`, `partsupp`, and `supplier` updates before propagating live join tuples to the aggregate. These checks are validation tests rather than a full performance benchmark. A complete benchmark would also record throughput, latency percentiles, parallelism, checkpoint configuration, JVM version, and machine resources. The performance argument therefore rests mainly on the Cquirrel papers, which report holistic multi-way join maintenance and $O(\lambda)$ amortized update cost for acyclic foreign-key joins with small- λ streams [1, 2].

6 Limitations and Threats to Validity

This evaluation validates correctness, not end-to-end performance. It checks generated outputs against independent references under selected stream settings, but it does not measure throughput, latency, checkpoint cost, state size, or recovery behavior. It also focuses on final-state validation. This is appropriate for checking convergence after insertions and deletions, but a stronger evaluation would replay emitted deltas or compare sampled intermediate states, especially for the Q16 sliding-window run whose final answer is empty after tuple expiration.

7 Conclusion

This report shows that Cquirrel is more than a GUI wrapper around Apache Flink. Its main technical value is the compilation of SPJA queries over acyclic foreign-key schemas into Flink operators that maintain query answers incrementally. The local evaluation covers two query categories: Q1 confirms single-relation aggregate maintenance, and Q16 confirms acyclic join aggregation with grouped supplier counts at scale factor 0.1. Overall, the release demonstrates a coherent SQL-to-Flink pipeline and a practical implementation of the acyclic foreign-key join maintenance algorithm, but it should still be viewed as a research demonstration rather than a production-ready stream processing platform.

8 Future Directions

Future work should document the supported SQL grammar, report unsupported constructs or cyclic join graphs more precisely, and add a benchmark harness over queries, scale factors, stream types, and Flink parallelism values. The runtime stack should also be modernized from Flink 1.11.2 and Java 8 to supported versions, with care taken around generated Scala/Flink API changes.

References

- [1] Qichen Wang, Chaoqi Zhang, Danish Alsayed, Ke Yi, Bin Wu, Feifei Li, and Chaoqun Zhan. Cquirrel: Continuous Query Processing over Acyclic Relational Schemas. *Proceedings of the VLDB Endowment*, 14(12):2667–2670, 2021.
<https://vldb.org/pvldb/vol14/p2667-wang.pdf>.
- [2] Qichen Wang and Ke Yi. Maintaining Acyclic Foreign-Key Joins under Updates. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1225–1239, 2020. <https://home.cse.ust.hk/~yike/sigmod20.pdf>.
- [3] Apache Flink. Stateful Stream Processing Documentation. <https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/stateful-stream-processing/>.
- [4] Apache Flink. Apache Flink: Stateful Computations over Data Streams.
<https://flink.apache.org/>.
- [5] Transaction Processing Performance Council. TPC Benchmark H Standard Specification, Revision 3.0.1.
https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf.